# Limpora Docs

## Limpieza, Seguridad y Claridad

# 1 Sobre el proyecto

## 1.1 Que es?

Limpora es una aplicacion y servicio web que conecta a clientes con autonomos especializados en servicios de limpieza, facilitando el contacto, la contratacion y la gestion de trabajos de forma clara, rapida y segura.

## 1.2 A quien va dirigido?

Limpora esta dirigido a:

- Autonomos que trabajan en el ambito de la limpieza y desean encontrar nuevos clientes.
- Personas que
  - buscan servicios de limpieza puntuales o temporales.
  - no pueden realizar tareas de limpieza en su hogar.
  - prefieren delegar las tareas de limpieza por comodidad o falta de tiempo.

## 1.3 Tipos de servicios

- Limpieza interna del hogar o local
- Limpieza externa (fachadas, patios, terrazas)
- Limpieza de piscinas
- Limpieza de vehiculos
- Limpieza de objetos o articulos de alto valor

## 1.4 Modelo de negocio

Limpora basa su modelo de negocio en la intermediacion de servicios de limpieza entre clientes y profesionales. La plataforma aplica una tarifa o comision en el momento en que se concreta una cita o contratacion entre usuarios.

Ademas, la empresa dispone de una red propia de profesionales de limpieza, lo que permite ofrecer servicios directos dentro de la aplicacion. Esto garantiza disponibilidad, calidad controlada y una experiencia uniforme para los clientes.

De esta forma, Limpora combina dos fuentes de ingresos:

- Comisiones por intermediacion entre clientes y autonomos registrados.
- Ingresos por servicios realizados por profesionales propios de la plataforma.

# 2 Tecnologias utilizadas

Para este proyecto optamos por las siguientes herramientas:

## 2.1 Frontend

El frontend de Limpora ha sido desarrollado utilizando un stack moderno basado en React y herramientas del ecosistema JavaScript actual. Estas tecnologias permiten construir una interfaz rapida, mantenible y escalable.

### 2.1.1 Vite

Vite es una herramienta de construccion y entorno de desarrollo para aplicaciones web modernas que permite una ejecucion rapida del proyecto y actualizacion instantanea durante el desarrollo.

#### Porque?

- Alta velocidad de arranque y compilacion
- Recarga en caliente (HMR) inmediata
- Configuracion sencilla
- Enfoque moderno para proyectos frontend
- Integracion directa con React

### 2.1.2 React

React es una biblioteca de JavaScript para construir interfaces de usuario basadas en componentes reutilizables y estado reactivo.

#### Porque?

- Estandar de facto en desarrollo frontend moderno
- Gran ecosistema y comunidad
- Arquitectura basada en componentes
- Facil mantenimiento y escalabilidad

- Amplia documentacion y soporte

### 2.1.3 Tailwind CSS

Tailwind CSS es un framework de estilos basado en clases de utilidad que permite diseñar interfaces de forma rapida sin escribir CSS tradicional complejo.

#### Porque?

- Desarrollo visual rapido
- Consistencia de estilos
- Diseno responsive sencillo
- Configuracion minima
- Alta personalizacion

### 2.1.4 Axios

Axios es un cliente HTTP basado en promesas que permite realizar peticiones al backend desde el navegador de forma sencilla y estructurada.

#### Porque?

- Simplifica las llamadas a la API
- Manejo de respuestas y errores
- Interceptores para autenticacion
- Estandar ampliamente utilizado en React

### 2.1.5 Mock Service Worker (MSW)

Mock Service Worker es una herramienta que permite simular respuestas de API en el navegador o en tests sin necesidad de un backend real.

#### Porque?

- Permite desarrollo sin backend disponible
- Facil testing del frontend

- Simulacion realista de API
- Independencia entre frontend y backend
- Facil reutilizacion de mocks

### 2.1.6 Zod

Zod es una biblioteca de validacion de datos basada en esquemas que permite definir y comprobar estructuras de datos en tiempo de ejecucion.

#### Porque?

- Validacion segura de datos y formularios
- Integracion con TypeScript
- Prevencion de errores en API
- Definicion clara de modelos
- Libreria estable y madura

### 2.1.7 Zustand

Zustand es una libreria ligera de gestion de estado para aplicaciones React que permite compartir datos entre componentes de forma simple.

#### Porque?

- Alternativa moderna a Redux
- API simple y directa
- Poco codigo y sin boilerplate
- Alto rendimiento
- Facil integracion con React

## 2.2 Backend

El backend de Limpora ha sido desarrollado con Node.js y Express, siguiendo una arquitectura basada en API REST. Este backend gestiona la logica de negocio, autenticacion, acceso a base de datos y comunicacion con servicios externos.

Se han utilizado librerias orientadas a seguridad, validacion, rendimiento y mantenibilidad, con el objetivo de garantizar un sistema robusto y escalable.

### 2.2.1 Node.js y Express

Node.js permite ejecutar JavaScript en el servidor, mientras que Express proporciona una estructura ligera para construir APIs REST.

#### Porque?

- Entorno unificado JavaScript en frontend y backend
- Alto rendimiento en aplicaciones web
- Arquitectura modular
- Amplio ecosistema
- Estandar en APIs REST

### 2.2.2 Base de datos MySQL

El sistema utiliza MySQL como base de datos relacional para almacenar usuarios, servicios, reservas y datos de la plataforma.

Se emplea el driver mysql2 para la conexion y ejecucion de consultas.

#### Porque?

- Base de datos relacional madura
- Soporte de transacciones
- Integridad de datos
- Estandar en aplicaciones web
- Buen rendimiento

### 2.2.3 Autenticacion y seguridad

El backend implementa autenticacion basada en tokens JWT y cifrado de contraseñas mediante bcrypt.

Ademas, se aplican multiples capas de seguridad:

- helmet: proteccion de cabeceras HTTP
- cors: control de origenes permitidos
- hpp: prevencion de parameter pollution
- express-rate-limit: limitacion de peticiones
- express-slow-down: mitigacion de ataques de fuerza bruta
- cookie-parser: gestion segura de cookies

#### Porque?

- Proteccion frente a ataques comunes web
- Control de acceso seguro
- Prevencion de abuso de API
- Cumplimiento de buenas practicas backend

### 2.2.4 Validacion de datos

Se utiliza express-validator para validar y sanitizar datos recibidos en peticiones HTTP antes de procesarlos.

#### Porque?

- Prevencion de datos invalidos
- Seguridad frente a inyecciones
- Control de formato en API
- Reduccion de errores en backend

### 2.2.5 Logging y monitorizacion

El sistema implementa logging estructurado con pino y pino-http, junto con morgan para registros de desarrollo.

#### Porque?

- Registro de peticiones y errores

- Analisis de fallos
- Monitorizacion del servidor
- Alto rendimiento en produccion

### 2.2.6 Servicios externos

El backend integra servicios externos para funcionalidades adicionales:

- firebase-admin: gestion de servicios Firebase
  - Autenticacion con Email y contraseñas
  - Email ya verificados
  - Facilidad para el usuario
- googleapis: integracion con APIs de Google
  - OAuth
- resend: envio de correos electronicos
  - Verificacion Email
  - Recuperacion de contraseñas
  - Subcripciones a blogs
  - Notificaciones
  - Facturas

#### Porque?

- Notificaciones y comunicacion con usuarios
- Integracion con servicios cloud
- Automatizacion de procesos
- Escalabilidad de funcionalidades

### 2.2.7 Documentacion de API

La API REST se documenta mediante swagger-jsdoc y swagger-ui-express, permitiendo visualizar y probar endpoints desde una interfaz web.

#### Porque?

- Documentacion interactiva
- Facil integracion frontend
- Estandar OpenAPI
- Mantenimiento de API

### 2.2.8 Herramientas de desarrollo

Se utilizan herramientas para mejorar el flujo de desarrollo:

- dotenv: gestion de variables de entorno
- nodemon: recarga automatica en desarrollo
- eslint-define-config: configuracion de linting

#### Porque?

- Configuracion segura
- Desarrollo rapido
- Codigo consistente
- Buenas practicas

## 2.3 Arquitectura

La aplicacion web Limpora sigue una arquitectura cliente-servidor separada en dos capas principales: frontend y backend. Ambos sistemas se comunican mediante una API REST expuesta por el backend y consumida por el frontend.

Esta separacion permite desacoplar la interfaz de usuario de la logica de negocio, facilitando el mantenimiento, la evolucion del sistema y la escalabilidad del servicio tanto en horizontal como en vertical.

El frontend, desarrollado en React, se ejecuta en el navegador del usuario y gestiona la experiencia visual, la navegacion y la interaccion con la plataforma. Todas las operaciones de datos se realizan mediante peticiones HTTP hacia la API.

El backend, desarrollado en Node.js con Express, centraliza la logica de negocio, autenticacion, validacion de datos y acceso a la base de datos MySQL. Tambien gestiona la integrac-

ion con servicios externos como envio de correos y servicios cloud.

La comunicacion entre capas sigue el modelo REST, utilizando solicitudes HTTP con respuestas en formato JSON. Esto permite:

- Independencia entre frontend y backend
- Posibilidad de multiples clientes (web, movil, admin)
- Escalabilidad independiente de cada capa
- Facil integracion con terceros
- Mantenimiento modular del sistema

A nivel de despliegue, la aplicacion se ejecuta en contenedores Docker orquestados mediante Docker Compose dentro de un servidor VPS en AWS. Nginx actua como proxy inverso, recibiendo el trafico HTTPS del dominio y redirigiendolo a los servicios internos correspondientes.

Esta arquitectura permite:

- Escalar el backend sin afectar al frontend
- Sustituir o actualizar componentes de forma aislada
- Desplegar nuevas versiones sin interrupciones
- Reproducir el entorno en otros servidores

Limpora se encuentra desplegado en un servidor VPS en la nube, accesible mediante un dominio propio. La infraestructura permite ejecutar tanto el backend como el frontend en un entorno controlado, escalable y accesible desde internet.

La aplicacion esta disponible en:

www.limpora.xyz

### 2.4.1 Hostinger

Hostinger es el proveedor utilizado para la gestion del dominio y servicios asociados de DNS.

#### Porque?

- Facilidad de gestion de dominios
- Precios accesibles
- Buena reputacion en hosting
- Panel de control sencillo
- Integracion rapida con VPS externos

### 2.4.2 AWS

Amazon Web Services (AWS) es el proveedor de infraestructura en la nube utilizado para alojar el servidor VPS donde se ejecuta la aplicacion.

El backend y los servicios del sistema se ejecutan en una instancia virtual configurada y administrada por el equipo de desarrollo.

#### Porque?

- Infraestructura cloud escalable
- Alta disponibilidad
- Control total del servidor
- Estandar profesional en despliegues
- Amplia documentacion y soporte

### 2.4.3 Docker

Docker se utiliza para contenerizar la aplicacion y sus servicios, permitiendo ejecutar el sistema en entornos aislados y reproducibles.

Mediante contenedores se empaquetan el backend, dependencias y configuraciones necesarias para su ejecucion.

#### Porque?

- Entornos consistentes entre desarrollo y produccion
- Despliegue reproducible
- Aislamiento de servicios
- Facil migracion de servidor
- Escalabilidad futura

### 2.4.4 Docker Compose

Docker Compose es una herramienta que permite definir y ejecutar aplicaciones formadas por multiples contenedores mediante un unico archivo de configuracion. En Limpora se utiliza para orquestar los servicios necesarios del sistema, como el backend y la base de datos, facilitando su ejecucion conjunta.

La configuracion de contenedores se define en el archivo:

./docker-compose.yml

#### Porque?

- Orquestacion sencilla de multiples servicios
- Ejecucion completa del sistema con un solo comando
- Configuracion declarativa y versionable
- Facil reproduccion de entornos
- Simplificacion del despliegue

### 2.4.5 GitHub Workflows

GitHub Workflows (GitHub Actions) es el sistema de integracion y automatizacion utilizado para ejecutar tareas automaticas sobre el repositorio de Limpora, como pruebas, construccion y despliegue.

Mediante flujos definidos en el repositorio, el sistema puede validar cambios en cada actualizacion del codigo y automatizar procesos del ciclo de desarrollo.

### Porque?

- Automatizacion de tareas de desarrollo
- Integracion continua (CI)
- Validacion automatica del codigo
- Despliegue automatizado
- Integracion directa con el repositorio

### Su forma de usuario

Se configuro el repositiorio con su propia cuenta en el VPS y se le dieron los permisos minimos necesarios para limpiar los cambios locales, bajar los cambios remotos y ejecutar docker compose con exito.

## 2.4.6 Nginx

Nginx es el servidor web y proxy inverso utilizado en el despliegue de Limpora. Se encarga de recibir las peticiones HTTP desde internet y redirigirlas a los servicios internos de la aplicacion, como el frontend y el backend.

Tambien gestiona aspectos clave del acceso publico, como el dominio, certificados SSL y distribucion de trafico hacia los contenedores correspondientes.

### Porque?

- Servidor web de alto rendimiento
- Proxy inverso para aplicaciones Node.js
- Gestion de dominio y rutas
- Soporte de HTTPS y certificados SSL
- Estandar en despliegues web profesionales

### 2.4.7 Certbot

Certbot es la herramienta utilizada para generar y gestionar certificados SSL gratuitos de Let's Encrypt en el servidor de Limpora. Permite habilitar conexiones seguras HTTPS en el dominio de la aplicacion mediante Nginx.

El sistema obtiene y renueva automaticamente los certificados, garantizando que las comunicaciones entre usuarios y la plataforma se realicen de forma cifrada.

#### Porque?

- Habilita HTTPS en el dominio
- Certificados SSL gratuitos y confiables
- Integracion directa con Nginx
- Renovacion automatica
- Cumplimiento de seguridad web moderna